



Module Code & Module Title
CC5009NI Cyber Security in Computing

Assessment Weightage & Type
60% Group Coursework 02

Year and Semester
2025/26 Autumn Semester

Group Name:			
SN	Student Name	College ID	University ID
1	Poonam Maharjan	NP01NT4A240140	24046427
2	Bishu Kumar Shah	NP01NT4A240144	24046371
3	Manshi Bhandari	NP01NT4A240190	24046412
4	Sajeek Basnet	NP01NT4A240143	24046549

Assignment Due Date: Wednesday, April 29, 2026

Assignment Submission Date: Wednesday, April 29, 2026

Word Count (Where Required): 2473

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked.

I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1. Introduction	1
1.2 Types of SQL injection	2
1.3 Technical Terminologies	3
1.4 Aim and Objectives	4
2. Background: The Landscape of Injection Flaws	5
2.1 Introduction	5
2.2 How Injection Actually Works?	5
2.3 The Main Flavors of Injection	6
2.4 The Real-World Impact	8
3. Demonstration	9
3.1 Testing Environment Setup and Tools	9
3.2 Phase 1 – Creating the Database and Table	10
3.3 Phase 2 – Inserting User Data	11
3.4 Phase 3 – Developing the Vulnerable Login Page	12
3.5 Phase 4 – Testing Normal Login Behavior	13
3.6 Phase 5 – Performing SQL Injection Attack	13
7. References	15

Table of Figures

Figure 1: SQL Injection (Saafan, 2026)	1
Figure 2: Types of SQL Injection	2
Figure 3: Mechanism for Injection (Geeks, 2025)	6
Figure 4: SQL Injection (Kant, 2025)	7
Figure 5: Command Injection (Beschokov, 2025)	7
Figure 6: XAMPP Control Panel running	9
Figure 7: Database and users table created in phpMyAdmin	10
Figure 8 User data inserted into the user's table	11
Figure 9: Vulnerable login page created using PHP	12
Figure 10: Successful login using valid credentials	13
Figure 11: SQL injection attack by passing login authentication	14

Table of Tables

Table 1: Technical Terminologies	3
Table 2 System Configuration	9
Table 3 Table Structure	10
Table 4 Data Inserted.....	11
Table 5: Explanation of Payload.....	14

1. Introduction

Web applications have become an important part of the digital landscape, providing a quick approach to accessing information and allowing work, businesses, and fields to expand their capacity to reach more users.

SQL injections are a technique that is executed by attackers, hackers, or threat actors to target the database and exploit vulnerabilities and flaws in web applications. It allows the attacker to breach confidentiality and integrity by accessing, modifying, or deleting the data using illegitimate commands and methods. (Nasareddin, et al., 2021)



Figure 1: SQL Injection (Saafan, 2026)

Frequently targeted and continuously attacked by threat actors on Web Applications due to their rapid production and broad attack surface, it is one of the most vulnerable environments. The spike in use of web applications throughout the year has led to spike on data and web traffic as well, for example, businesses migrated their system previously offline to online, causing threat actors to exploit the system (Paul, et al., 2024).

The Open Worldwide Application Security Project (OWASP) has listed SQL Injection on top five security risks on the latest 2025 list. As previously, it was on top three security risk (OWASP TOP 10, 2025).

1.2 Types of SQL injection

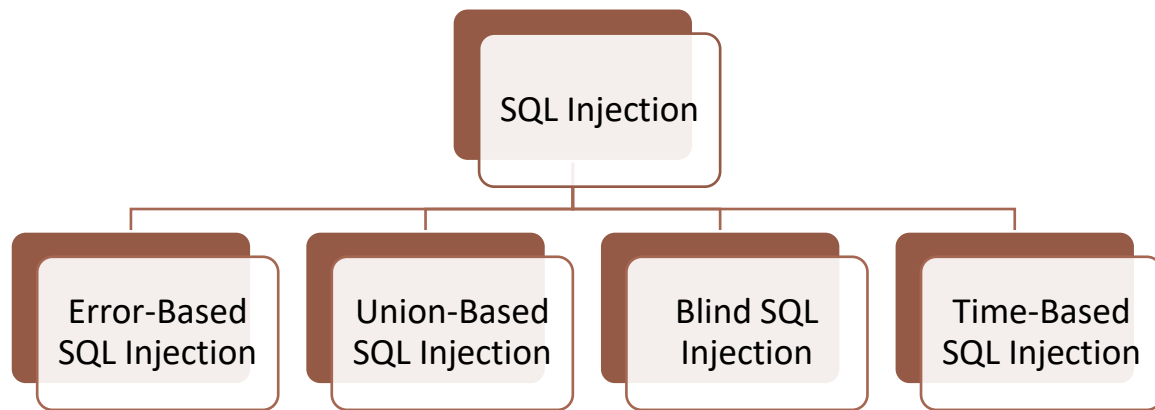


Figure 2: Types of SQL Injection

Union-Based SQL Injection: Union Query Attacks are attacks that use the UNION operator to join the output or results which is obtained from a legit with evil queries, providing the attacker with a path into other table data or information.

In simple words, this technique uses the SQL UNION operator, which helps in combining the result of a malicious query with the original or legitimate one.

This allows the attacker to extract data from other tables, but the attacker must know the structure of the database.

Error-Based SQL Injections: It is a type of SQL injection where information is obtained about the structure of the database by using an error message which is provided by the database server.

Blind SQL Injection: Blind SQL Injection attacks do not provide immediate results or feedback from their application, but it shows the behavior or structure and content of the database by analyzing statements.

In simple words, the application does not directly show query results, but attackers send queries and observe certain behavior or structure, and later the data is extracted indirectly.

This method is used for retrieving sensitive information, such as user passwords, private data or confidential credentials, also known as Data Exfiltration.

(Sanghavi & Samdani, 2024)

Time-based SQL Injection: It is a subtype of blind SQL injection, where it sends a set of true or false questions to the database and based on the application response it determines the result. It also seems same as classical SQL injection, but it is slightly different on how the data is retrieved from the database and is considered more difficult as the database does not output the data to the webpage. (Nasareddin, et al., 2021)

1.3 Technical Terminologies

Technical Terms	Definitions
Confidentiality	It is defined as the process of keeping sensitive information secret and confidential. If the confidential fails, the data is leaked or it is exposed.
Integrity	Integrity means, the data stays unchanged and stays as it is until and unless the authorized user changes. If unauthorized user access and changes, modifies or tempers, the integrity fails.
Attack Surface	Attack surface is understood as all the possible vectors or ways an attacker can attack and try to break into the system. If there is more attack surface in the system, it has more risk for the system.
OWASP Top 10	It is the list of most critical web security risks listed by the OWASP, which is a non-profitable organization.
Data Exfiltration	The process of stealing data from a system, computer or by various cyberattack methods is known as Data Exfiltration.

Table 1: Technical Terminologies

1.4 Aim and Objectives

This coursework aims to research SQL injection attack, injections vulnerabilities in web applications, and understand its backgrounds, how it can be exploited, its mitigation, preventive methods and application areas.

Objectives

The objective of this coursework is as follows:

- Research on SQL injection attacks, vulnerabilities or weaknesses which makes SQL injections possible.
- Analyzing the different types of injection attack.
- Understanding the background of injection flaws in web application.
- Demonstrating SQL injection attack in a controlled and safe environment.
- Testing different scenarios to mitigate the issue or flaws.
- Providing advantages and disadvantages of mitigation technique.
- Understanding the application areas.

2. Background: The Landscape of Injection Flaws

2.1 Introduction

Web applications are built to face the public and constantly interact with users, they naturally sit directly in the crosshairs of security threats. Among these, injection flaws have been a massive, ongoing headache for the internet. The Open Web Application Security Project (OWASP) has consistently ranked them as a top-tier risk for years.

Why do they keep happening? It boils down to the sheer complexity of modern web development, combined with the fact that many developers simply haven't received specialized security training (Crespo-Martinez, 2020). Web apps constantly act as the "middleman" between users and critical backend systems like databases and operating systems. Unfortunately, the connecting points between these systems are exactly where attackers like to strike.

2.2 How Injection Actually Works?

Think of an injection flaw as a dangerous miscommunication. It happens when an application takes input from a user but fails to properly double-check or "sanitize" it before passing it along to the backend (Halfond & Orso, 2005).

In a perfectly secure app, anything a user types into a form are treated strictly as plain data. But when an app is vulnerable, the backend system gets confused between what is *harmless data* and what is an executable command. This confusion allows an attacker to slip in malicious code disguised as normal input. When the backend (like a database engine) reads it, it gets tricked into running the attacker's code as if it were a legitimate instruction (Hoang Viet Long, 2020).

Even though the security community understands this problem incredibly well, it persists. Developers often rely on weak, outdated defenses—like simply creating a "blacklist" of bad characters—which attackers can easily outsmart with creative new variations (Halfond & Orso, 2005).

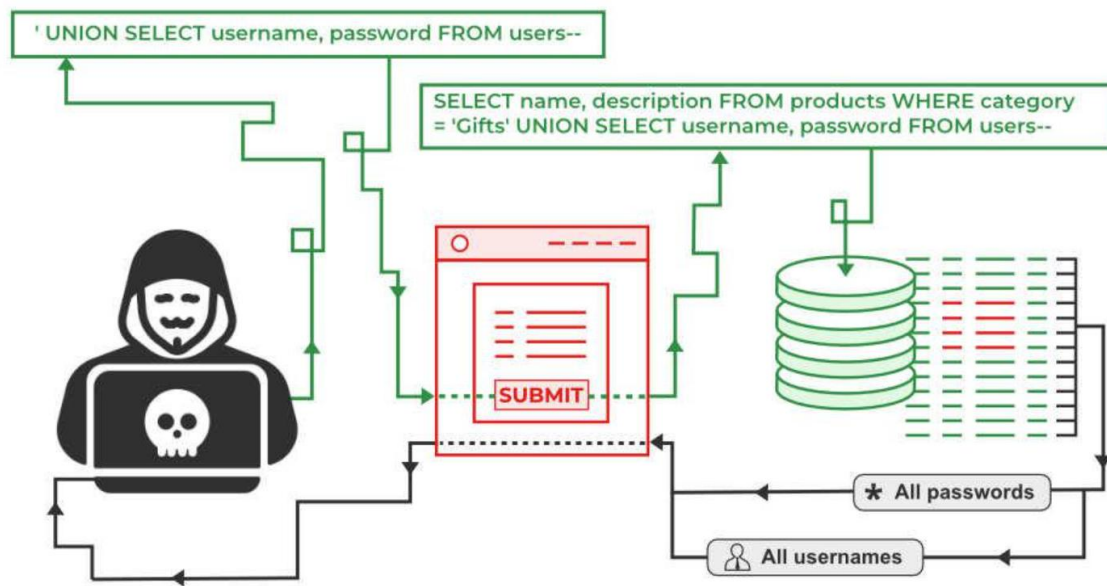


Figure 3: Mechanism for Injection (Geeks, 2025)

2.3 The Main Flavors of Injection

"Injection" is an umbrella term, and the exact nature of the attack changes depending on what the hacker is targeting. The two biggest culprits include:

- SQL Injection (SQLi):** This happens when an app mishandles user input that is headed for a database. By tweaking something as simple as a login form or a website URL, an attacker can sneak malicious SQL commands right into the backend (Halfond & Orso, 2005). Since databases hold the "crown jewels" of any application—like user passwords, financial records, and private data, SQLi remains one of the most popular and damaging attacks out there.



Figure 4: SQL Injection (*Kant, 2025*)

- Command / OS Injection:** Instead of going after the database, this attack targets the server's actual operating system. If an app passes user input straight to system-level commands (which happens often in environments like Node.js) without keeping it in a safe sandbox, an attacker can force the server to run arbitrary operating system commands (Cristian-Alexandru Staicu, 2018).

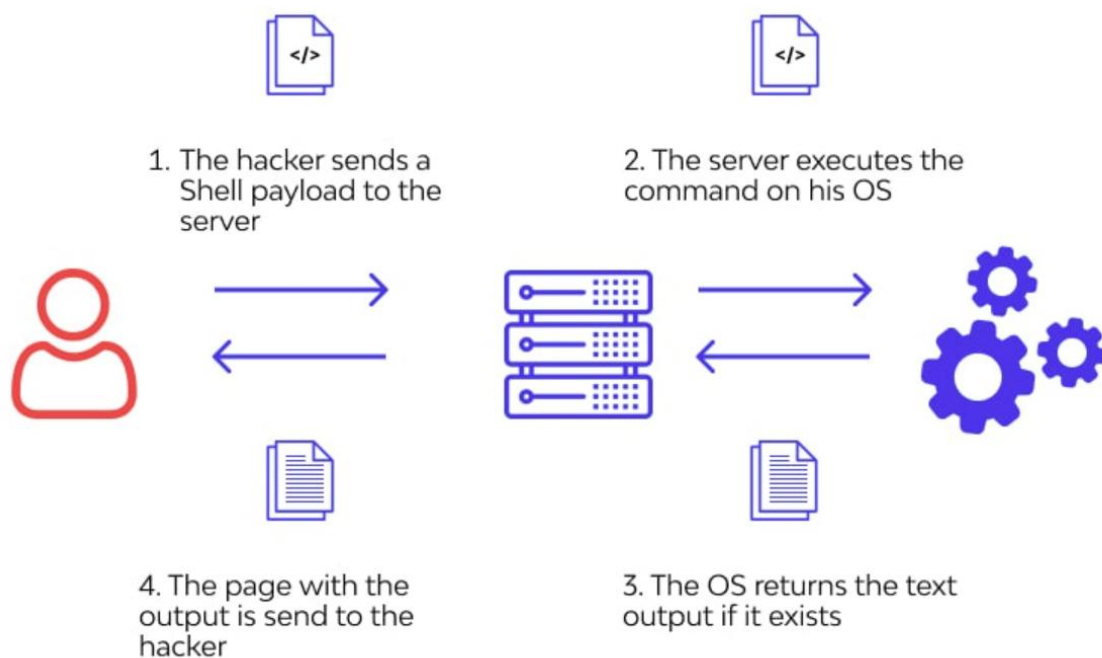


Figure 5: Command Injection (*Beschokov, 2025*)

2.4 The Real-World Impact

The fallout from a successful injection attack is usually disastrous for a business. Because these flaws let attackers hijack the application's intended logic, they can often just step right over passwords and normal security checkpoints.

With SQL injections, this usually results in highly sensitive data being leaked, stolen, or destroyed (Hoang Viet Long, 2020). Command injections are often even scarier; it allows attackers to break out of the application entirely. Once they do that, they can mess with the server's file system, cause massive crashes, or completely take over the host machine, turning it into a playground for further attacks (Cristian-Alexandru Staicu, 2018).

3. Demonstration

3.1 Testing Environment Setup and Tools

The SQL injection vulnerability was tested within a local controlled environment using XAMPP, which offers the Apache web server and MySQL database that are needed to execute the web app. The system has been developed in-house to make sure that all testing operations have been done safely without any interference with the outer networks and real users.

Component	Description
Server Platform	XAMPP
Programming Language	PHP
Database	MySQL
Database Name	Dragon_db
Table Name	Users
Operating System	Windows
Development Tool	Visual Studio Code

Table 2 System Configuration

The vulnerable login software was developed on PHP and MySQL to model a real-world authentication system. The application lets users type in a username and password, which are matched to the database records. Nevertheless, the login system was deliberately developed without adequate input validation in order to show how SQL injection attacks can be done.

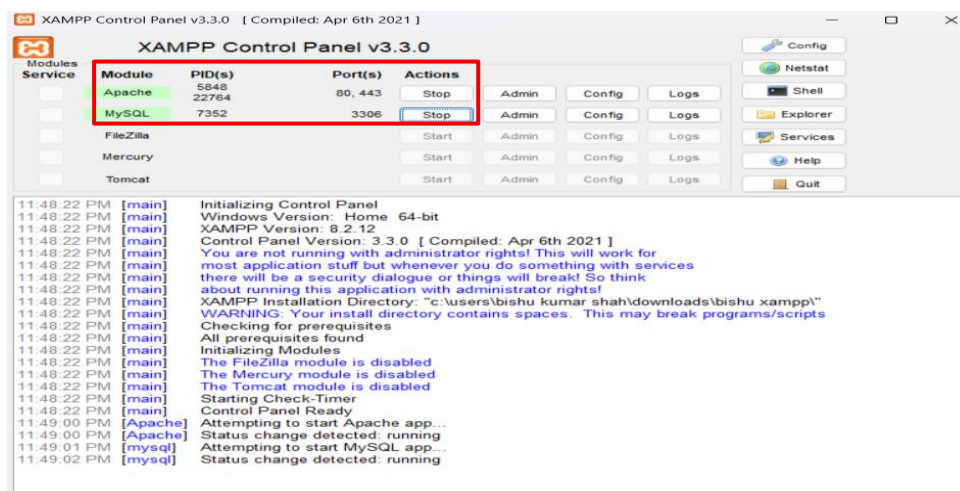


Figure 6: XAMPP Control Panel running

3.2 Phase 1 – Creating the Database and Table

The demonstration was initiated by developing a database and table that would hold user credentials. This was done through the phpMyAdmin interface that comes with XAMPP.

Field	Type	Description
Id	INT	Unique user identifier
Username	VARCHAR	Stores username
Password	VARCHAR	Stores password

Table 3 Table Structure

A new database was created, which is called dragon_db, and then a table was created with three fields, which are id, username, and password. The ID field was defined as a primary key and auto-incremented to give a new value with every new record.

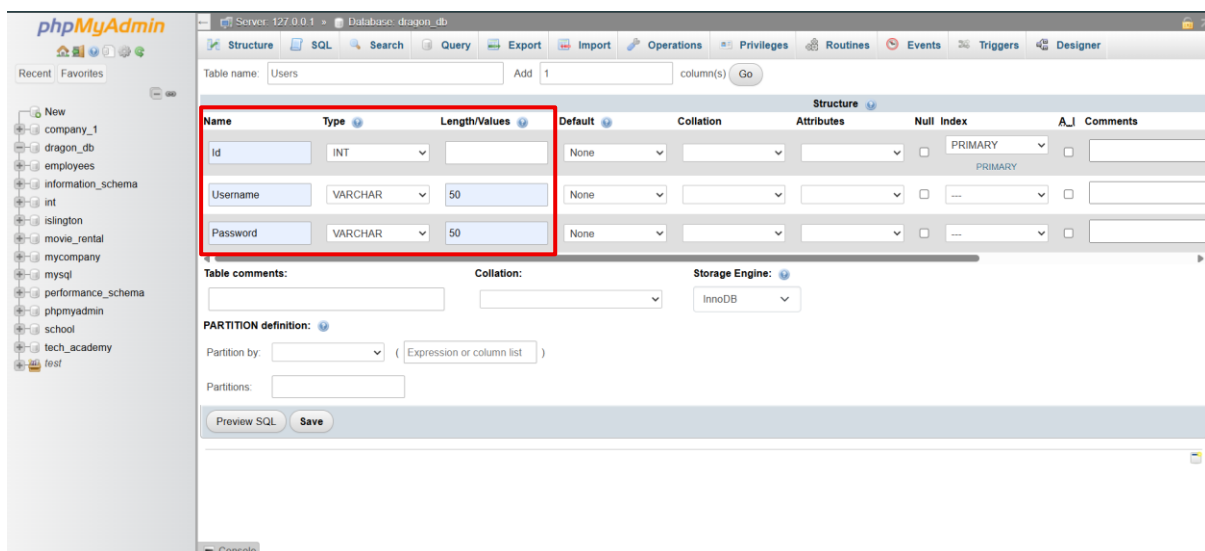


Figure 7: Database and users table created in phpMyAdmin

This structure represents a simple user authentication database commonly used in web applications.

3.3 Phase 2 – Inserting User Data

After creating the table, sample user records were inserted into the database. These records were used to test the login functionality and demonstrate how SQL injections can manipulate database queries.

Username	Password
admin	admin123
user1	pass123

Table 4 Data Inserted

These records simulate normal user accounts in a web application.

The screenshot shows the phpMyAdmin interface for a database named 'dragon_db'. The 'users' table is selected, and its structure is displayed. The table has two columns: 'Id' (int(11)) and 'Username' (varchar(50)). The 'Password' column is not visible in the structure view. The data entry form is shown, with the following values entered:

Column	Type	Function	Null	Value
Id	int(11)			
Username	varchar(50)			admin
Password	varchar(50)			admin123

The 'Go' button is visible next to the password field. Below the structure view, the 'Insert as new row' option is selected, and the 'Go' button is visible next to the 'Preview SQL' button.

Figure 8 User data inserted into the user's table

3.4 Phase 3 – Developing the Vulnerable Login Page

A login page named **login.php** was created using PHP. This page connects to the database and checks whether the entered username and password match any record in the users table.

The following SQL query was used to validate user credentials:

```
SELECT * FROM users WHERE username='$username' AND password='$password';
```

This query directly includes user input without validation or parameterization. As a result, it is vulnerable to SQL injection attacks.

When a user enters login credentials, the application sends the input directly to the database for verification.

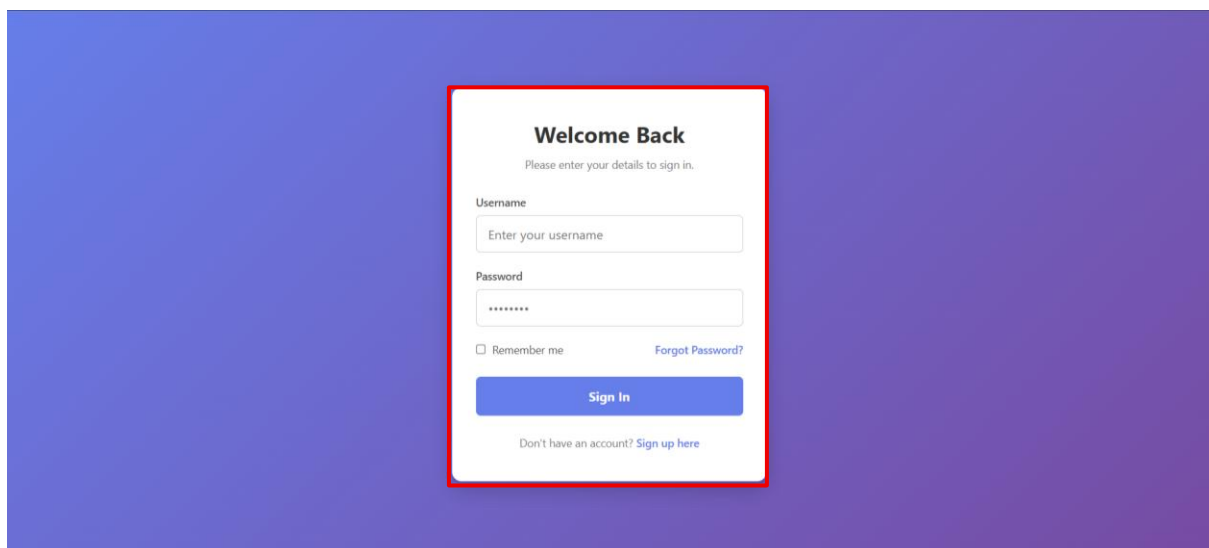


Figure 9: Vulnerable login page created using PHP

3.5 Phase 4 – Testing Normal Login Behavior

Before performing the SQL injection attack, the login system was tested using valid credentials to confirm that it was functioning correctly.

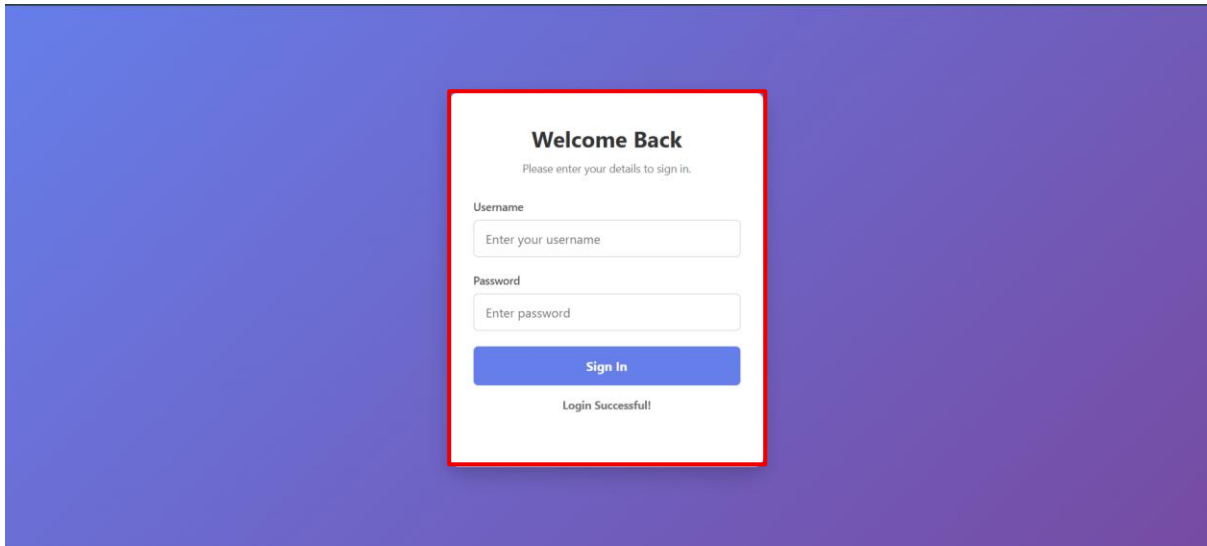


Figure 10: Successful login using valid credentials

This confirmed that the login system was correctly retrieving user data from the database.

3.6 Phase 5 – Performing SQL Injection Attack

After confirming normal login behavior, a SQL injection payload was entered into the login form to bypass authentication.

Payload Used

Username: ' OR '1'='1

Password: pass123

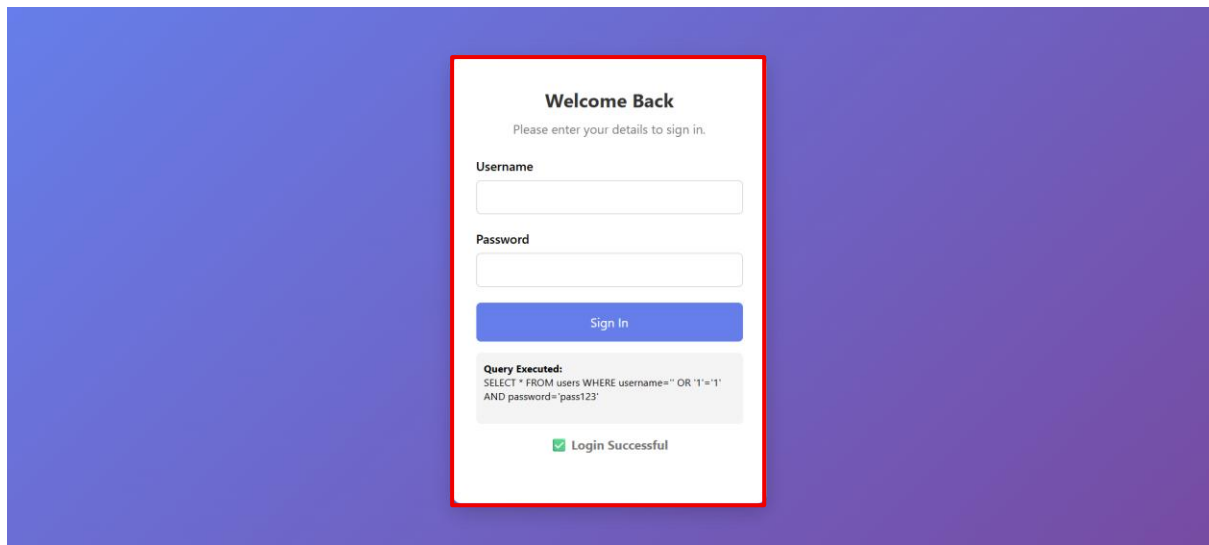


Figure 11: SQL injection attack by passing login authentication

Component	Meaning
'	Close the original SQL string
OR	Logical operator
'1'='1	Always true condition

Table 5: Explanation of Payload

7. References

- Beschokov, M., 2025. *What is a Command Injection?*. [Online] Available at: <https://www.wallarm.com/what/command-injection-definition>
- Crespo-Martinez, E., 2020. *Vulnerability analysis with SQLMAP*. [Online] Available at: <https://ingenius.ups.edu.ec/ingenius/article/view/25.2021.10>
- Cristian-Alexandru Staicu, M. P. B. L., 2018. *Synode: Understanding and Automatically Preventing Injection Attack on Node.js*, Darmstadt, Germany: TU Darmstadt and Imperial College London.
- Geeks, G. f., 2025. *SQL Injection*. [Online] Available at: <https://www.geeksforgeeks.org/sql/sql-injection/>
- Halfond, W. G. J. & Orso, A., 2005. *AMNESIA: Analysis and Monitoring for NEutralizing SQL-Injection Attacks*, Atlanta, GA: Georgia Institute of Technology.
- Hoang Viet Long, T. A. T. D. T. N. V. C. H. M. H. N. T. K. S., 2020. *An efficient algorithm and tool for detecting dangerous website vulnerabilities*. [Online] Available at: <https://dl.acm.org/doi/abs/10.1504/ijwgs.2020.106128>
- Kant, A., 2025. *SQL Injection: All Concepts, All Payloads, All In One*. [Online] Available at: <https://dev.to/deoxys/sql-injection-all-concepts-all-payloads-all-in-one-4ch5>
- Nasareddin, M., Alkhamaiseh, A., Qasaimeh, M. & Al-Qassas, R., 2021. A systematic review of detection and prevention techniques of SQL injection attacks. *Information Security Journal: A Global Perspective*, 32(4), pp. 252-265.
- OWASP TOP 10, 2025. *OWASP TOP 10*. [Online] Available at: https://owasp.org/Top10/2025/A05_2025-Injection/ [Accessed 12 April 2026].
- Paul, A., Sharma, V. & Olukoya, O., 2024. SQL injection attack: Detection, prioritization & prevention. *Journal of Information Security and Applications*, Volume 85, pp. 103871-103871.
- Saafan, A., 2026. *nilebits*. [Online] Available at: <https://www.nilebits.com/blog/2024/06/sql-injection-understanding-the->

[threat/?utm_source=Pinterest&utm_medium=organic](#)

[Accessed 12 April 2026].

Sanghavi, K. & Samdani, K., 2024. Overview of Sql Injection Types, Defense Tools and Prevention Techniques. *Tuijin Jishu/Journal of Propulsion Technology*, 45(4), pp. 2780-2796.

William G. J. Halfond, A. O., 2005. *AMNESIA: analysis and monitoring for NEutralizing SQL-injection attacks*. [Online] Available at: <https://dl.acm.org/doi/10.1145/1101908.1101935>